FILE INPUT / OUTPUT IN C

- Programming in C

Dr. Saurabh Singh Raghuvanshi,
Asst Prof, SoCS, UPES Dehradun

Learning outcomes

By the end of this lesson

Explain why programs use files instead of relying only on keyboard input and screen output.

Identify the FILE pointer and common file-opening modes.

Open, read, write, and close text files using standard C functions.

Check for errors and avoid common file-handling mistakes.

Build a small program that stores data persistently.

Why file handling?

Keyboard input and screen output are temporary.

Files allow data to remain available after a program ends.

Common uses: student records, reports, configuration files, logs, and data exchange.

A file workflow usually follows: open → process → close.

Always decide whether the program should read, create, replace, or append data.

The basic file-handling model

Core idea: a FILE pointer represents a stream managed by the C library

Include the standard I/O header: #include <stdio.h>

Declare a file pointer: FILE *fp;

Open a file and check the result: fp = fopen("data.txt", "r");

Use suitable functions to read from or write to the file.

Close the file when finished: fclose(fp);

Opening modes at a glance

Choose modes carefully: w can erase previous contents

r — open an existing file for reading; fails if it does not exist.

w — create a file for writing; an existing file is truncated.

a — append to a file; create it if it does not exist.

r+ — read and write an existing file.

w+ — read and write; create or truncate the file.

a+ — read and append; create the file if needed.

Binary modes add b where relevant, e.g., rb or wb.

Writing text to a file

Common output functions

fprintf(fp, ...): formatted output, similar to printf.

fputs(text, fp): writes a string.

fputc(ch, fp): writes one character.

Check whether opening succeeded before writing.

Close the file to flush buffered output and release resources.

Example 1 — write to a file

Creates/replaces message.txt and writes two lines

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("message.txt", "w");

    if (fp == NULL) {
        perror("Could not open file");
        return 1;
    }

    fprintf(fp, "Hello, file!\n");
    fprintf(fp, "C file handling practice.\n");

    fclose(fp);
    return 0;
}
```

Reading text from a file

Common input functions

fgetc(fp): reads one character; returns EOF at end or on read error.

fgets(buffer, size, fp): reads at most size − 1 characters and adds '\0'.

fscanf(fp, ...): reads formatted data; check the number of items read.

For lines of text, fgets is usually safer than an unbounded string read.

Test the return value of each read operation.

Example 2 — read a file line by line

Reads each line safely and prints it to the screen

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("message.txt", "r");
    char line[100];

    if (fp == NULL) {
        perror("Could not open file");
        return 1;
    }

    while (fgets(line, sizeof line, fp) != NULL) {
        printf("%s", line);
    }

    if (ferror(fp)) {
        perror("Error while reading");
    }

    fclose(fp);
    return 0;
}
```

Character-by-character processing

Use int for the result of fgetc(), not char, so EOF can be represented.

A loop continues while the result is not EOF.

Useful for counting characters, spaces, tabs, and newline characters.

Distinguish end-of-file from a read error when correctness matters.

Example 3 — count characters

Counts bytes/characters as read from a text stream; text-mode translation may apply

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("message.txt", "r");
    int ch;
    long count = 0;

    if (fp == NULL) {
        perror("Could not open file");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF) {
        count++;
    }

    if (ferror(fp)) {
        perror("Read error");
        fclose(fp);
        return 1;
    }

    printf("Characters: %ld\n", count);
```

Formatted data: fprintf and fscanf

Example format strings are teaching examples; validate real input

fprintf(fp, "%d %s", id, name) writes formatted fields.

fscanf(fp, "%d %29s", &id, name) reads fields into variables.

For strings, set a width limit in the format to avoid buffer overflow.

Check the return value: fscanf returns the number of successfully assigned items.

For names containing spaces, use line-based input and parse deliberately.

Example 4 — save and load a record

Output: Roll: 101, Marks: 86

```
#include <stdio.h>

int main(void) {
    FILE *fp;
    int roll = 101, marks = 86;
    int r, m;

    fp = fopen("result.txt", "w");
    if (fp == NULL) {
        perror("Open for writing");
        return 1;
    }
    fprintf(fp, "%d %d\n", roll, marks);
    fclose(fp);

    fp = fopen("result.txt", "r");
    if (fp == NULL) {
        perror("Open for reading");
        return 1;
    }
    if (fscanf(fp, "%d %d", &r, &m) == 2)
        printf("Roll: %d, Marks: %d\n", r, m);
    else
```

Appending instead of replacing

Use mode "a" when new data should be added at the end.

Opening in "w" truncates an existing file; do not use it for appending.

For repeated records, write one complete record per line.

Check fopen() and fclose() results in robust applications.

Example 5 — append a log entry

Each run adds a new line without removing earlier entries

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("activity.log", "a");

    if (fp == NULL) {
        perror("Could not open log");
        return 1;
    }

    if (fprintf(fp, "Program executed successfully\n") < 0) {
        perror("Write error");
        fclose(fp);
        return 1;
    }

    if (fclose(fp) == EOF) {
        perror("Close error");
        return 1;
    }
    return 0;
}
```

Binary files: the big picture

Use text files for easy inspection; binary formats need a defined layout

Binary files store data as bytes rather than human-readable text.

fread() reads blocks of bytes; fwrite() writes blocks of bytes.

Open binary files with modes such as "rb" and "wb".

Check the number of elements actually read or written.

Writing a C structure's raw bytes can be non-portable due to padding, representation, and endianness.

Binary I/O pattern

Pattern only: ensure fp is open in a compatible mode and items has enough space

```
size_t written = fwrite(items, sizeof items[0], count, fp);
if (written != count) {
    /* Handle a short write or error */
}

rewind(fp);  /* return to the beginning */

size_t read_count = fread(items, sizeof items[0], count, fp);
if (read_count < count) {
    if (ferror(fp)) {
        /* Handle read error */
    } else if (feof(fp)) {
        /* Reached end of file */
    }
}
```

File positioning and status

Check function return values; positioning rules vary for text and binary streams

rewind(fp): move to the beginning of the stream and clear its error/EOF indicators.

fseek(fp, offset, origin): move to a position; origin is SEEK_SET, SEEK_CUR, or SEEK_END.

ftell(fp): obtain the current file position when supported.

feof(fp): tests whether an earlier read encountered end-of-file; it does not predict EOF.

ferror(fp): tests whether a stream error indicator is set.

A robust file-handling checklist

Good habits matter as much as remembering function names

Use a meaningful file name and the correct mode.

Check fopen() against NULL before using the pointer.

Validate every read/write result that affects correctness.

Close every successfully opened file on all appropriate paths.

Do not assume a file exists in the program's current working directory.

Avoid unsafe input patterns and validate data read from files.

Common mistakes to avoid

Using a file pointer after fopen() failed.

Opening an existing data file with "w" by accident.

Using while (!feof(fp)) as the read loop condition.

Ignoring fscanf(), fread(), fprintf(), or fclose() return values.

Forgetting that fscanf("%s", ...) needs a maximum field width.

Assuming raw binary data is portable across machines or compiler settings.

Practice problems

Try each task first, then test normal and error cases

Beginner: write your name, roll number, and course to a text file.

Beginner: read a text file and display it line by line.

Intermediate: count lines, words, and characters in a text file.

Intermediate: append student marks as one record per line.

Advanced: copy one file to another and report read/write errors.

Challenge: design a simple student-record file format and validate each record.

Quick knowledge check

Answers: NULL • a • EOF is set only after a read attempts past the end • line input vs formatted parsing • ferror() and feof()

1. What does fopen() return if it cannot open a file?
2. Which mode appends data without truncating earlier content?
3. Why is while (!feof(fp)) a poor read-loop pattern?
4. What is the difference between fgets() and fscanf()?
5. Which functions help distinguish a read error from end-of-file?

Summary

C file I/O uses FILE streams and functions from <stdio.h>.

Typical sequence: fopen → read/write → check status → fclose.

Select modes carefully: r, w, a, and their update/binary variants.

Prefer bounded input and check function return values.

Practice with small files before building record-oriented programs.

Reference

Reference link: pdvpmtasgaon.edu.in • Let Us C, Fifth Edition

Reference: Yashavant P. Kanetkar, Let Us C, Fifth Edition, linked PDF.